CodeMentor AI — Wie es funktioniert

Statische Analyse + Lokale Compiler + Dual-Pass LLM-Architektur (Schichtenbericht K0 - K6)

7-SCHICHTEN-PIPELINE | DETERMINISTISCHER KERN | 0% LÖSUNGS-LEAKAGE | 33 ADR-STANDARDS

1. Architektur-Ablaufdiagramm



2. Detaillierte Funktionsweise der Schichten

Schicht 0: Eingabe & Normalisierung (K0)

Zeilenenden und Zeichenkodierungen werden bereinigt. Programmiersprache und Kontext (z.B. Unity/Unreal) werden automatisch identifiziert. Eingaben über 50 KB werden abgewiesen.

Schicht 1: Vorvalidierung & Früher Abbruch (K1)

Tier 0 (tree-sitter): Findet Syntaxfehler direkt im Parse-Baum (`ERROR/MISSING`). Bei Syntaxfehlern bricht die Analyse sofort ab (~1 ms), **ohne LLM-Aufruf**.

Tier 1 (Lokale Compiler): Falls gcc, clang oder dotnet verfügbar sind, erfolgt eine reine Syntaxprüfung (`-fsyntax-only`). Der Code wird niemals ausgeführt.

Schicht 2: Deterministische statische Analyse (K2)

AST-Abfragen extrahieren Fakten: Verschachtelungstiefe, Aufrufgraph und dynamische Speicherallokationen. 41 statische Regeln identifizieren bekannte Schwachstellen. Die Big-O-Komplexität wird deterministisch berechnet.

Schicht 3: Sichere Prompt-Erstellung (K3)

Benutzercode wird strikt als nicht vertrauenswürdig behandelt und in `<user_code>`-Tags isoliert, um Prompt-Injection zu verhindern. Fakten aus K2 werden dem LLM übergeben, was Halluzinationen verhindert.

Schichten 4 & 5: Dual-Pass LLM & Leakage Guard (K4 - K5)

- **Durchlauf A (Tiefenanalyse):** Ermittelt logische Schwachstellen und Randfälle. Wird gegen strikte Zeilenbeweise und JSON-Schemas validiert.
- **Durchlauf B (Sokratisches Mentoring):** Formuliert die Befunde in didaktische Fragen um.
- **Leakage Guard:** Überprüft alle Fragen. Wird versehentlich die fertige Lösung verraten, wird die Frage verworfen und neu generiert (0% Leakage).

Schicht 6: Multi-Format-Berichte (K6)

Erzeugt Ausgaben in JSON, Markdown, SARIF 2.1.0 sowie in der modernen React Web- und Desktop-Oberfläche.

3. Progressives Hinweissystem

Hinweisstufe	Pädagogischer Fokus	Typischer Mentoring-Ansatz
Stufe 1: Konzeptuell	Allgemeine Mechanismen	"Welche Lebensdauer hat dynamisch allozierter Speicher nach Verlassen der Funktion?"
Stufe 2: Zeilenkontext	Konkrete Variablen & Zeilen	"Wurde der Zeiger `tmp` nach Abschluss der Schleife in Zeile 8 freigegeben?"
Stufe 3: Zielgerichtet	Konkreter Lösungsweg (kein Code)	"Denken Sie an die Standard-C-Funktion zur Speicherfreigabe vor der Rückgabeanweisung."